

FaaS Performance Evaluation Project

Usecase: Resume/JD Matcher API

Tools which are going to be used:

- **Docker – to avoid the problem of ‘it works on my PC’**
 - Here we simply use a docker container and create an instance (most probably an image) and deploy/build our project on it. In that way, we can prevent the local machine support issues.
- **Kubernetes -** If Docker is the "Box," Kubernetes (K8s) is the **Warehouse Manager**.
 - Kubernetes can be used to make a cluster and increase the number of replicas (simply the number of containers can be increased inside a cluster, and we can see different results of performance)

//before containerizing, we can check and test locally in our PC for the correct functioning of Azure functions.

Phase 1: Local Development (Azure Functions)

Objective: Build the HTTP-triggered API and test the NLP logic locally.

1. Initialize the project:

```
func init FaaSTestProject --python -m V2
```

2. Install dependencies:

```
pip install -r requirements.txt
```

(Ensure scikit-learn and azure-functions are listed).

3. Run the function locally:

```
func start
```

4. Test the endpoint (New Terminal):

```
curl -X POST http://localhost:7071/api/match -H "Content-Type: application/json" -d '{"resume": "Python dev", "jd": "Python developer"}
```

Phase 2: Containerization (Docker)

Objective: Package the application into a portable container image.

1. Build the image (Platform-specific for Mac M-series):

```
docker build --platform linux/amd64 -t resume-matcher-v1
```

2. Run and test the container:

```
docker run -p 8080:80 resume-matcher-v1
```

3. Debug if it crashes (Exit Code 134):

```
docker ps -a (to find ID)
```

```
docker logs <container_id>
```

```
Docker kill <container_id> //if it is needed to stop the container running
```

Phase 3: Orchestration (Minikube & Kubernetes)

Objective: Deploy and scale the API within a local cluster to test "System Reliability" and "Scalability". Here I use minikube.

1. Start the cluster:

```
minikube start --driver=docker
```

2. Load local image into Minikube:

```
minikube image load resume-matcher-v1
```

3. Apply the configuration:

```
kubectl apply -f deployment.yaml
```

4. Check Pod status (Watch for CrashLoopBackOff):

Wait 10 s before running this code, because it needs some time to start.

```
kubect1 get pods
```

5. Scale to 3 Replicas (for the Scaling test):

Wait 10s before running this code, because it needs some time to start. Here, I try to do the scaling part with 3 replicas. According to our needs, this number can be changed.

```
kubect1 scale deployment resume-matcher-deploy --replicas=3
```

6. Expose the service:

(Keep this terminal open separately for the URL, then we can change the curl command with this ip address instead of localhost)

```
minikube service resume-matcher-service --url
```

Then, we can test this curl,

```
curl -X POST http://localhost:7071/api/match -H "Content-Type: application/json" -d '{"resume": "Python dev", "jd": "Python developer"}
```

<http://localhost:7071/api/match> ---> can be replaced with the given IP from above command

//this is to be done.

Phase 4: Benchmarking & Evaluation

Objective: Collect "Response Latency" and "Cold vs. Warm Start" data.

1. Automated Benchmark (Using your script):

```
chmod +x benchmark.sh
```

```
./benchmark.sh
```

2. View Internal Cluster Logs (To see NLP processing):

```
kubectl logs -l app=resume-matcher
```

3. Describe Pods (To debug resource/memory bottlenecks):

```
kubectl describe pod <pod_name>
```

//already did this part and to be continued all the time

Phase 5: Version Control (GitHub)

Objective: Maintain "Deployment via GitHub" as per the proposal.

1. Initialize Git:

```
git init
```

2. Create and Add .gitignore (Critical first step):

```
touch .gitignore (Add local.settings.json, __pycache__, etc.)
```

```
git add .gitignore
```

3. Commit and Push:

```
git commit -m "Initial commit: Set up infrastructure and K8s manifests"
```

```
git remote add origin <repo_url>
```

```
git push -u origin main
```

Here afterward, let's commit the changes from our own dev branches and let's merge to main branch after reviewing them together.

The things can be carried out

- Enhance the NLP model to have more accurate results
- Enhance the input type (for resume) for the model.
- Automating the evaluation process
- Analyze several results with graphs

- If there is time, make a simple front end to enter Resume and JD to be more user friendly.